

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical 5

FastAPI Backend and Security Suite

Name	Chakrika Panakala
Roll No.	2520090042
Course	25CS1302E
CO Mapping	CO3
Session	Practical 5

Aim

To build a FastAPI 'Gatekeeper' for BookFlow with a Pydantic BookSchema (13-character ISBN, positive price, no future published_date), an async POST /books route that saves to PostgreSQL with SQLAlchemy 2.x async and logs a background task, an async pgvector search endpoint using <=>, and a member registration route that hashes passwords with passlib bcrypt, protected by a reusable dependency-injected security guard.

Program

Installation and Run Commands

```
pip install fastapi uvicorn "sqlalchemy[asyncio]" asyncpg pgvector \ "passlib[bcrypt]" bcrypt==4.0.1 pyjwt uvicorn
main:app --reload --port 8000
```

main.py – Configuration and Imports

```
import logging import os import re from contextlib import asynccontextmanager from datetime import date, datetime,
timedelta, timezone from decimal import Decimal from typing import AsyncIterator, List, Optional import jwt from fastapi
import BackgroundTasks, Depends, FastAPI, HTTPException, Query, status from fastapi.security import
OAuth2PasswordBearer, OAuth2PasswordRequestForm from passlib.context import CryptContext from pgvector.sqlalchemy import
Vector from pydantic import BaseModel, ConfigDict, EmailStr, Field, field_validator from sqlalchemy import Integer,
Numeric, String, select, text from sqlalchemy.exc import IntegrityError from sqlalchemy.ext.asyncio import AsyncSession,
async_sessionmaker, create_async_engine from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column DATABASE_URL =
os.getenv( "DATABASE_URL", "postgresql+asyncpg://postgres:postgres@localhost:5432/bookflow_db" ) JWT_SECRET =
os.getenv( "JWT_SECRET", "change-this-secret-in-production" ) JWT_ALGORITHM = "HS256" TOKEN_MINUTES = 30 EVENT_LOG_FILE =
os.getenv( "EVENT_LOG_FILE", "book_events.log" ) logging.basicConfig( level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s" ) logger = logging.getLogger( "bookflow" ) engine = create_async_engine( DATABASE_URL, pool_size=10,
max_overflow=20 ) SessionLocal = async_sessionmaker( engine, expire_on_commit=False ) pwd_context =
CryptContext( schemes=[ "bcrypt" ], deprecated="auto" ) oauth2_scheme = OAuth2PasswordBearer( tokenUrl="/auth/login" )
```

SQLAlchemy 2.x ORM Models

```
class Base(DeclarativeBase): pass class Book(Base): __tablename__ = "books" book_id: Mapped[int] =
mapped_column(Integer, primary_key=True) title: Mapped[str] = mapped_column(String(200), nullable=False) isbn:
Mapped[str] = mapped_column(String(13), unique=True, nullable=False) published_year: Mapped[Optional[int]] =
mapped_column(Integer) published_date: Mapped[Optional[date]] = mapped_column(Date) price: Mapped[Optional[Decimal]] =
mapped_column(Numeric(10, 2)) embedding: Mapped[Optional[List[float]]] = mapped_column(Vector(3)) class Member(Base):
__tablename__ = "members" member_id: Mapped[int] = mapped_column(Integer, primary_key=True) full_name: Mapped[str] =
mapped_column(String(100), nullable=False) email: Mapped[str] = mapped_column(String(150), unique=True, nullable=False)
password_hash: Mapped[Optional[str]] = mapped_column(String(255))
```

Pydantic Schemas and Validation

```
class BookSchema(BaseModel): title: str = Field(min_length=1, max_length=200) isbn: str = Field(min_length=13,
max_length=13) price: Decimal = Field(gt=0, max_digits=10, decimal_places=2) published_date: date
@field_validator("isbn") @classmethod def isbn_must_be_digits(cls, value: str) -> str: if not value.isdigit(): raise
ValueError("isbn must contain only digits") return value @field_validator("published_date") @classmethod def
date_not_in_future(cls, value: date) -> date: if value > date.today(): raise ValueError("published_date cannot be in the
future") return value class BookOut(BaseModel): model_config = ConfigDict(from_attributes=True) book_id: int title: str
isbn: str price: Optional[Decimal] = None published_date: Optional[date] = None class SearchHit(BaseModel): book_id: int
title: str cosine_distance: float class MemberRegister(BaseModel): full_name: str = Field(min_length=2, max_length=100)
email: EmailStr password: str = Field(min_length=8, max_length=72) class MemberOut(BaseModel): model_config =
ConfigDict(from_attributes=True) member_id: int full_name: str email: str class Token(BaseModel): access_token: str
token_type: str = "bearer"
```

Embedding Helper

The three theme dimensions match the VECTOR(3) embeddings used in Practical 4: space/science fiction, computer science, and self-improvement. The helper converts free text into a normalized three-dimensional theme vector.

```
THEME_KEYWORDS = ( {"space", "planet", "mars", "alien", "astronaut", "galaxy", "star", "desert", "sci-fi"}, {"code",
"software", "database", "algorithm", "data", "programming", "sql", "system"}, {"habit", "habits", "focus",
"productivity", "life", "mind", "success", "work"}, ) def embed_text(query: str) -> List[float]: words =
re.findall(r"[a-z\-\-]+", query.lower()) scores = [0.1 + sum(word in keywords for word in words) for keywords in
THEME_KEYWORDS] total = sum(scores) return [round(score / total, 4) for score in scores]
```

Dependency Injection and Security Guard

```
async def get_db() -> AsyncIterator[AsyncSession]: async with SessionLocal() as session: yield session async def
get_current_member( token: str = Depends(oauth2_scheme), db: AsyncSession = Depends(get_db) ) -> Member: unauthorized =
HTTPException( status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid or expired token", headers={"WWW-Authenticate":
"Bearer"}, ) try: payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM]) member_id = int(payload["sub"])
except (jwt.PyJWTError, KeyError, ValueError): raise unauthorized member = await db.get(Member, member_id) if member is
None: raise unauthorized return member
```

Background Task

```
def log_new_book_event(book_id: int, title: str, member_email: str) -> None: timestamp =
datetime.now(timezone.utc).isoformat(timespec="seconds") with open(EVENT_LOG_FILE, "a", encoding="utf-8") as handle:
handle.write( f"{timestamp} | New Book Added | id={book_id} | " f"title={title} | by={member_email}\n" )
```

Application Startup and Authentication Routes

```
@asynccontextmanager async def lifespan(_: FastAPI) -> AsyncIterator[None]: async with engine.begin() as conn: await
conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector")) await conn.execute(text( "ALTER TABLE books ADD COLUMN IF
NOT EXISTS price NUMERIC(10,2)" )) await conn.execute(text( "ALTER TABLE books ADD COLUMN IF NOT EXISTS published_date
DATE" )) await conn.execute(text( "ALTER TABLE books ADD COLUMN IF NOT EXISTS embedding VECTOR(3)" )) await
conn.execute(text( "ALTER TABLE members ADD COLUMN IF NOT EXISTS password_hash VARCHAR(255)" )) logger.info("Database
schema ready") yield await engine.dispose() app = FastAPI(title="BookFlow API", version="1.0.0", lifespan=lifespan)
@app.post("/auth/register", response_model=MemberOut, status_code=201) async def register(payload: MemberRegister, db:
AsyncSession = Depends(get_db)) -> Member: member = Member( full_name=payload.full_name, email=payload.email.lower(),
password_hash=pwd_context.hash(payload.password), ) db.add(member) try: await db.commit() except IntegrityError: await
db.rollback() raise HTTPException(409, "Email is already registered") await db.refresh(member) return member
@app.post("/auth/login", response_model=Token) async def login(form: OAuth2PasswordRequestForm = Depends(), db:
AsyncSession = Depends(get_db)) -> Token: result = await db.execute( select(Member).where(Member.email ==
form.username.lower()) ) member = result.scalar_one_or_none() if member is None or not member.password_hash or not
pwd_context.verify( form.password, member.password_hash ): raise HTTPException(401, "Incorrect email or password")
expires = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_MINUTES) token = jwt.encode( {"sub":
str(member.member_id), "exp": expires}, JWT_SECRET, JWT_ALGORITHM ) return Token(access_token=token)
```

Protected POST /books Route

```
@app.post("/books", response_model=BookOut, status_code=201) async def create_book( payload: BookSchema,
background_tasks: BackgroundTasks, db: AsyncSession = Depends(get_db), member: Member = Depends(get_current_member), )
-> Book: book = Book( title=payload.title, isbn=payload.isbn, price=payload.price,
published_date=payload.published_date, published_year=payload.published_date.year, embedding=embed_text(payload.title),
) db.add(book) try: await db.commit() except IntegrityError: await db.rollback() raise HTTPException(409, "A book with
this ISBN already exists") await db.refresh(book) background_tasks.add_task( log_new_book_event, book.book_id,
book.title, member.email ) return book
```

Async pgvector Search Endpoint

```
@app.get("/books/search", response_model=List[SearchHit]) async def search_books( q: str = Query(min_length=2,
max_length=200), limit: int = Query(default=3, ge=1, le=50), db: AsyncSession = Depends(get_db), ) -> List[SearchHit]:
query_vector = embed_text(q) distance = Book.embedding.cosine_distance(query_vector) statement = ( select(Book.book_id,
Book.title, distance.label("cosine_distance")) .where(Book.embedding.is_not(None)) .order_by(distance) .limit(limit) )
rows = (await db.execute(statement)).all() return [ SearchHit( book_id=r.book_id, title=r.title,
cosine_distance=round(r.cosine_distance, 4) ) for r in rows ]
```

API Test Commands

```
BASE=http://127.0.0.1:8000 # 1. Register a member curl -X POST $BASE/auth/register \ -H 'Content-Type: application/json'
\ -d '{"full_name":"Chakrika Panakala","email":"chakrika.api@kluniversity.in","password":"Secure@2026"}' # 2. Login and
receive a JWT TOKEN=$(curl -s -X POST $BASE/auth/login \ -d
'username=chakrika.api@kluniversity.in&password=Secure@2026' \ | python3 -c 'import
sys,json;print(json.load(sys.stdin)["access_token"])') # 3. Wrong password curl -X POST $BASE/auth/login \ -d
'username=chakrika.api@kluniversity.in&password=wrongpass' # 4. POST /books without a token curl -X POST $BASE/books \
-H 'Content-Type: application/json' \ -d '{"title":"Hitchhiker Guide to the
Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}' # 5. POST /books with a valid token curl
-X POST $BASE/books \ -H "Authorization: Bearer $TOKEN" \ -H 'Content-Type: application/json' \ -d '{"title":"Hitchhiker
Guide to the Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}' # 6. Validation test curl -X
POST $BASE/books \ -H "Authorization: Bearer $TOKEN" \ -H 'Content-Type: application/json' \ -d '{"title":"Bad
Book","isbn":"12345","price":-50,"published_date":"2030-01-01"}' # 7. Semantic search curl -G $BASE/books/search \
--data-urlencode 'q=stories about space and planets' -d limit=3 # 8. Database/software semantic search curl -G
$BASE/books/search \ --data-urlencode 'q=learn database and software code' -d limit=3 # 9. Background task log sleep 1
cat book_events.log # 10. Verify stored password hashes psql -c "SELECT email, LEFT(password_hash, 29) || '...' AS
password_hash FROM members WHERE password_hash IS NOT NULL;"
```

Output

1. FastAPI server startup

Uvicorn starts successfully, the application performs the database startup checks, and the API runs on `http://127.0.0.1:8000`.

Test	Expected Result
Member registration	HTTP 201 – member created; password stored as bcrypt hash
Successful login	HTTP 200 – JWT access token returned
Wrong password	HTTP 401 – Incorrect email or password
POST /books without token	HTTP 401 – Not authenticated
POST /books with valid token	HTTP 201 – book created and background task scheduled
Invalid ISBN / price / future date	HTTP 422 – validation errors
Space semantic search	HTTP 200 – space-related books ranked by cosine distance
Software/database search	HTTP 200 – technical books ranked by cosine distance

Representative API Results

Registration: HTTP 201 with `member_id`, `full_name` and email returned. The password is not returned.

Login: HTTP 200 with a bearer access token.

Unauthenticated POST /books: HTTP 401 with “Not authenticated”.

Authenticated POST /books: HTTP 201 with the created book, including its title, ISBN, price and published date.

Validation: HTTP 422 for an invalid ISBN, negative price and future publication date.

Semantic Search Results

Query	Ranked Books
stories about space and planets	Hitchhiker Guide to the Galaxy (0.0000); The Martian (0.0008); Dune (0.0009)
learn database and software code	Designing Data-Intensive Applications (0.0001); Introduction to Algorithms (0.0001); Database Systems (0.0001)

Background Task Output

After the book creation response, `book_events.log` records a “New Book Added” entry containing the book ID, title and authenticated member email.

Stored Password Hash

The members table stores a bcrypt hash beginning with a `$2b$` prefix rather than the plain-text password. This verifies that password storage uses hashing.

Result

The FastAPI BookFlow Gatekeeper was implemented with Pydantic validation, asynchronous SQLAlchemy 2.x database operations, pgvector cosine search, background event logging, bcrypt password hashing, JWT authentication and a reusable dependency-injected security guard. Invalid requests were rejected through validation and protected routes required authentication.

Conclusion

The experiment demonstrates how a FastAPI backend can connect PostgreSQL, pgvector and security components into a single asynchronous service. The completed API validates book data, protects resource creation with JWT-based authentication, performs semantic search and records background events while keeping passwords stored as secure hashes.